

LIBERO-RECOVER: BEYOND TASK SUCCESS TOWARDS FAILURE RECOVERY IN ROBOTIC MANIPULATION MODELS

Lin Liu, Lu Zhang✉ & Huchuan Lu

School of Information and Communication Engineering
Dalian University of Technology

Wu Yang, Shuai Tao & Wulong Liu

Beta Infinity

Ziying Song✉

Nanyang Technological University

Zhicheng Bao

Beijing Jiaotong University

ABSTRACT

Vision-Language-Action (VLA) or World Action (WAM) models have recently demonstrated remarkable performance in robotic manipulation. On LIBERO, SOTA method have achieved nearly 100% success rates, seemingly suggesting that the models are ready for deployment in real world. However, near perfect performance on existing benchmarks can be misleading: success under ideal conditions does not imply real world robustness. Existing benchmarks primarily evaluate task completion from predefined initial states, while real world interactions inevitably involve failures such as failed grasps, collisions, and unintended object movements. A robot must therefore not only execute tasks successfully, but also recognize and recover from failures to continue the task. Yet this capability remains largely unmeasured, revealing a critical gap between benchmark performance and real world reliability. To address this gap, we introduce LIBERO-Recover Benchmark, a large scale benchmark for failure recovery in robotic manipulation. Built upon LIBERO, we collect real execution failures from SOTA embodied models and construct 1,000+ scenarios across four recovery levels: (1) Action Retry, (2) Action Adaptation, (3) Object State Recovery, and (4) Environmental Recovery. We evaluate four core capabilities: spatial understanding, object structure reasoning, interaction understanding, and topological reasoning. As the first large-scale benchmark for embodied failure recovery, LIBERO-Recover shifts evaluation from *Can the robot succeed?* to *Can the robot recover after failure?*, promoting robust and generalizable embodied agents. The project will be available in <https://liulin815.github.io/LIBERO-Recovery/>.

1 INTRODUCTION

VLAs and WAMs have achieved remarkable progress in robotic manipulation. On early benchmarks such as LIBERO Liu et al. (2023), SOTA models have approached 100% success rates, with some recent works even omitting LIBERO results Liu et al. (2023), arguing that its tasks are no longer sufficiently challenging to differentiate model capabilities. Yet, from LIBERO Liu et al. (2023) and SimplerEnv Li et al. (2024) to RoboTwin2.0 Chen et al. (2025) and RoboCasa Nasiriany et al. (2024), the prevailing evaluation paradigm remains largely unchanged: given a predefined initial state, models are evaluated by whether they can complete a task in a single attempt. This paradigm largely assumes idealized execution, whereas real world manipulation inevitably involves failures such as unsuccessful grasps, collisions, and unintended object displacement. Robust robots must therefore not only complete tasks, but also recognize failures, recover disrupted states, replan, and resume execution.

Recent benchmarks have extended LIBERO toward more challenging and realistic settings. LIBERO-Pro Zhou et al. (2025) introduces more complex task variations, LIBERO-Plus Fei et al. (2025) evaluates robustness across multiple dimensions, LIBERO-X Wang et al. (2026) considers

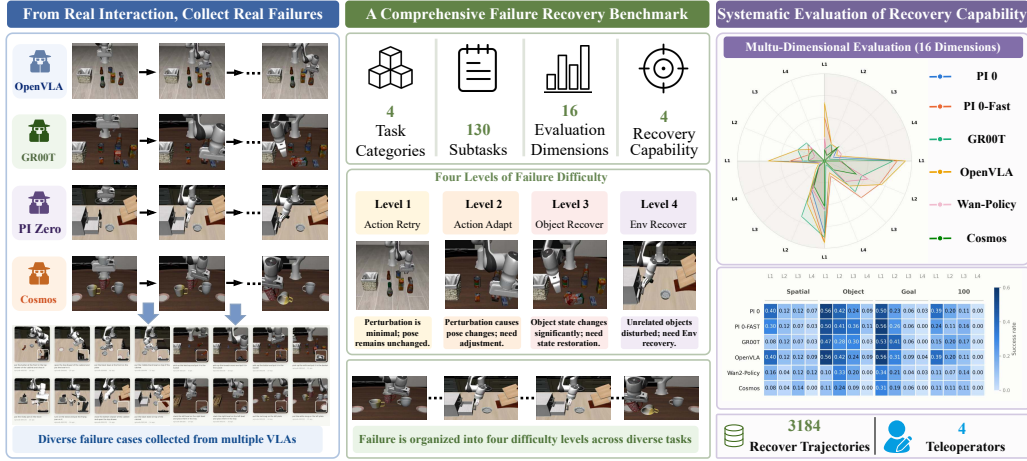
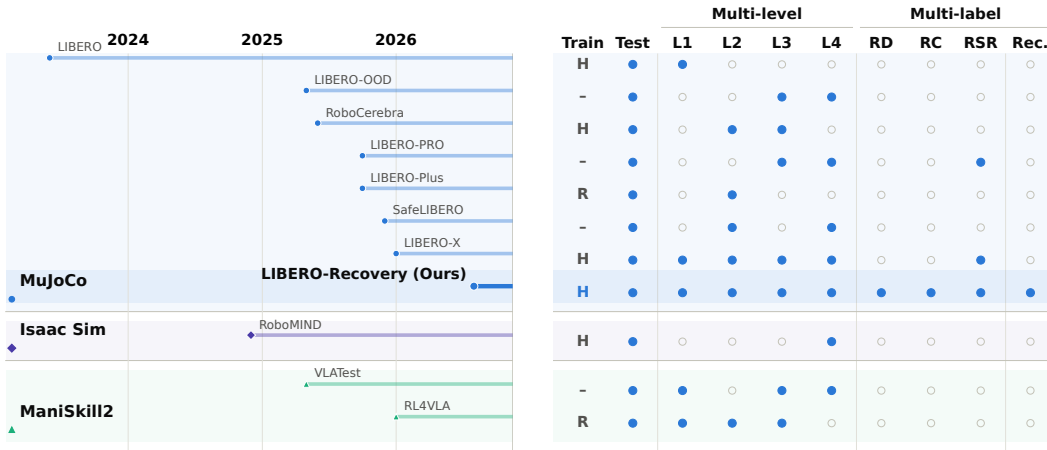


Figure 1: **Overview of LIBERO-Recover.** LIBERO-Recover collects failure scenarios from SOTA models and expert recovery trajectories via human teleoperation. The radar chart reveals existing models’ limitations in failure recovery.

variations in task and environmental conditions, and LIBERO-Safety Cui et al. (2026) focuses on safety-critical failures and undesired interactions. However, these benchmarks still primarily evaluate performance under predefined perturbations or challenging conditions, leaving failure recognition, state recovery, and post-failure replanning largely unexplored. We argue that failure recovery is a fundamental capability for robust and practically deployable robotic manipulation.

However, existing benchmarks typically construct test scenarios by manually altering the initial positions or appearances of objects, rather than simulating failures that actually arise during task execution and their resulting consequences. As a result, such evaluations primarily assess a model’s generalization to environmental variations, rather than its ability to understand state changes caused by real failures, plan appropriate recovery actions, and resume task execution. To address this gap, we propose LIBERO-Recover, an extension benchmark specifically designed to evaluate failure recovery in embodied models. Unlike existing benchmarks that construct test environments by manually perturbing object poses, appearances, or scene layouts, LIBERO-Recover naturally collects failure states and their consequences arising from real embodied models. LIBERO-Recover develops an evaluation protocol covering four core capabilities: embodiment understanding, object structure reasoning, topological interaction reasoning, and failure recovery, across 130 subtasks, and 16 evaluation dimensions. Experiments show that existing VLA models’ failure recovery capabilities remain substantially limited, revealing a significant gap between high task success rates and robustness under real-world interactions. Our key contributions are summarized as follows.

- We show that standard LIBERO primarily evaluates one-shot task success under idealized conditions, potentially overestimating the robustness of embodied models to real-world failures.
- We introduce **LIBERO-Recover**, a failure recovery benchmark built from naturally occurring failures during embodied model interactions, covering **4 task categories, 130 subtasks, and 16 evaluation dimensions** to assess four core recovery capabilities.
- We evaluate leading VLA models and reveal a substantial performance gap between standard task execution and failure recovery, highlighting critical limitations in failure-state understanding and recovery planning.



3 LIBERO-RECOVER BENCHMARK

3.1 PROBLEM FORMULATION

We consider a manipulation task specified by a language instruction l , with a goal condition $\mathcal{G}$. Starting from an initial state s_0 , a manipulation policy π_θ generates a sequence of actions based on the robot’s observations, resulting in an execution trajectory:

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_T). \quad (1)$$

Conventional Task Execution. Under the conventional evaluation setting, the robot is expected to continuously execute the task from a valid initial configuration until reaching a goal-satisfying state. The desired execution therefore follows,

$$s_0 \rightarrow s_1 \rightarrow \dots \rightarrow s_T, \quad (2)$$

with the terminal state satisfying:

$$\mathcal{G}(s_T, l) = 1. \quad (3)$$

This setting primarily measures whether a policy can produce an action sequence that successfully completes the task without explicitly considering how to proceed after an execution failure.

Failure Recovery. Our setting instead considers an execution that deviates from the intended task progression and reaches an intermediate state s_f . We refer to s_f as a *failure state* when the original task goal is not satisfied at this point,

$$\mathcal{G}(s_f, l) = 0, \quad (4)$$

but the task remains recoverable through subsequent interaction. The key difference is that the policy can no longer simply continue the original action sequence. Instead, it must determine how the preceding failure has altered the current task configuration and generate a new sequence of actions that restores the necessary state before continuing toward the original goal. We therefore decompose the execution into a pre-failure trajectory and a recovery trajectory:

$$s_0 \rightarrow \dots \rightarrow s_f \rightarrow s_{f+1} \rightarrow \dots \rightarrow s_T, \quad (5)$$

where the post-failure actions constitute a recovery policy π_{rec} . A failure state is considered *recoverable* if:

$$\mathcal{G}(s_f, l) = 0 \quad \text{and} \quad \exists, \pi_{\text{rec}} \text{ s.t. } \mathcal{G}(s_T, l) = 1. \quad (6)$$

Failure recovery requires robots to understand, adapt, and recover from unexpected states.

3.2 FAILURE OR STATE VARIATION ?

Not every state change during execution constitutes a failure. We define a state s_f as a failure state when the preceding action induces a task-relevant change that invalidates the current execution plan:

$$\mathcal{F}(s_t, a_t, s_f) \iff \underbrace{s_t \xrightarrow{a_t} s_f}_{\text{execution-induced}} \wedge \underbrace{\mathcal{G}(s_f, l) = 0}_{\text{task-relevant}} \wedge \underbrace{\pi_\theta(s_f) \not\models \mathcal{G}}_{\text{plan-invalidating}}. \quad (7)$$

Here, the last condition means that directly continuing the current execution is insufficient to complete the original task, requiring adaptation or recovery. A valid failure state must nevertheless remain recoverable,

$$\exists \pi_{\text{rec}} \text{ s.t. } s_f \xrightarrow{\pi_{\text{rec}}} s_T, \quad \mathcal{G}(s_T, l) = 1. \quad (8)$$

Thus, state changes that preserve task executability are treated as normal variations, whereas execution-induced changes that invalidate the current plan and require recovery are treated as failures.

3.3 FAILURE TAXONOMY AND DIFFICULTY LEVELS

We organize recoverable failures into four progressive difficulty levels according to the amount of state reasoning and recovery required to resume task execution, as illustrated in Fig. 3.



Figure 3: **Four recovery difficulty levels of LIBERO-Recover.** Each level demands progressively deeper state reasoning, from re-executing a failed action (L_1) to restoring environmental states that block task completion (L_4).

L_1 Action Retry. The failure does not materially alter the task configuration, i.e., $s_f \approx s_{\text{expected}}$. Recovery therefore only requires retrying the failed action, such as reattempting a grasp when the target object remains unchanged.

L_2 Action Adaptation. The failure causes a limited change in the object configuration, such that $s_f \neq s_{\text{expected}}$, but the object remains directly usable. The model must adapt its action to the observed state rather than blindly repeating the failed action:

$$a' = \pi(o_f, l) \neq a_{\text{failed}}. \quad (9)$$

L_3 Object State Recovery. The failure substantially alters a task-relevant object, making direct task completion impossible. The model must first restore the object’s required state and then resume the original task. For example, after a failed grasp displaces a bowl, the robot must first recover the bowl’s pose before continuing the instructed manipulation.

L_4 Environmental Recovery. The failure affects an object or state that is not directly manipulated by the original task but subsequently prevents task execution. Recovery therefore requires reasoning about both task-relevant and task-irrelevant states and their interaction topology. The model must identify the environmental obstruction, restore the relevant configuration, and then continue the original task.

Overall, the hierarchy progresses from *action repetition*, to *action adaptation*, to *task-relevant object recovery*, and finally to *environment recovery*, providing increasingly demanding tests of state- and topology-aware failure recovery.

3.4 RECOVERY SCENARIO CONSTRUCTION

Scenario Definition. Rather than manually perturbing task states to synthesize failures, LIBERO-Recover grounds every scenario in a failure that actually occurred during embodied model execution. Formally, each recovery scenario is defined as

$$(\mathcal{I}, s_0, \tau_{\text{fail}}, s_f, g, r), \quad (10)$$

where $\mathcal{I}$ denotes the task instruction, s_0 the original initial state, τ_{fail} the failure trajectory, s_f the resulting failure state, g the original task goal, and r the required recovery behavior. A scenario is thus not a static perturbation of a task state, but a task instance grounded in an actual failure transition and its associated recovery requirement.

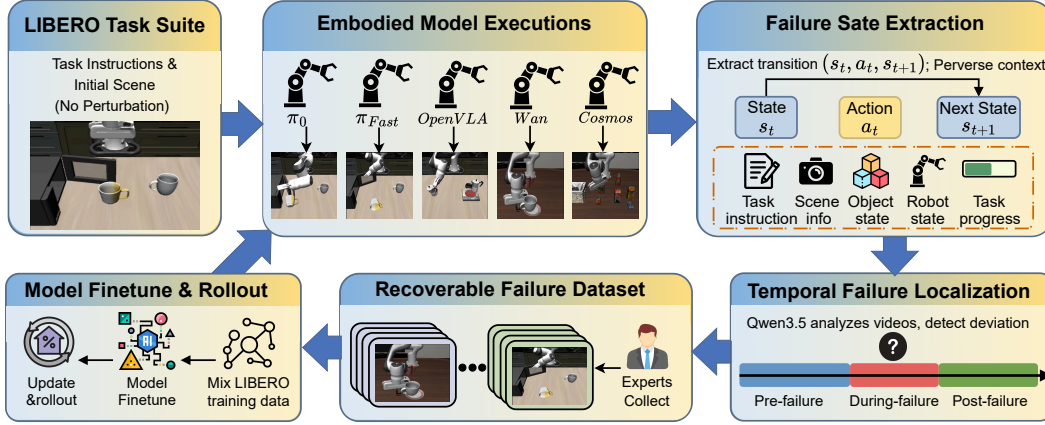


Figure 4: The construction pipeline of LIBERO-Recover.

Construction Pipeline. As illustrated in Fig. 4, each scenario is constructed through a three-stage pipeline, with the source tasks, failure generators, and annotation models introduced stage by stage.

Stage 1: Task Execution. We start from four LIBERO task suites covering 130 subtasks, preserving their original instructions and initial configurations. Multiple embodied policies, including π_0 , $\pi_{0.5}$, OpenVLA, GR00T, Wan-Policy, and Cosmos-Policy, are deployed as *failure generators*. Each execution from s_0 produces a complete failure trajectory τ_{fail} , and the simulator states are rendered into videos for subsequent temporal analysis. No objects, environment states, or initial configurations are manually perturbed, and no failures are injected during execution.

Stage 2: Failure Localization. For each execution video, we employ Qwen3.5-27B-Instruct to analyze the entire execution together with the task instruction and to temporally localize where the execution deviates from the intended progression. Each video is decomposed into three stages,

$$V = V_{\text{pre}} \cup V_{\text{fail}} \cup V_{\text{post}}, \quad (11)$$

corresponding to the execution before, during, and after the failure-causing interaction. Based on the predicted temporal boundaries, we retrieve the corresponding robot simulation states and object poses directly from the simulator, and extract the localized failure scenario.

Stage 3: Failure Characterization. For each candidate failure scene, we render its execution trajectory into a video and provide the resulting video, together with the corresponding task instruction, to Qwen3.5-27B-Instruct. The model is then prompted to assess the post-failure consequences and determine the corresponding recovery type and difficulty level according to the taxonomy defined in Sec. 3.3.

3.5 BENCHMARK STATISTICS

The resulting benchmark contains **2178** recovery scenarios derived from **130** subtasks across **four** task categories, covering **four** recovery levels and **16** evaluation dimensions. In total, failures are generated by **three** embodied models. The overall benchmark statistics are summarized in Fig. 5. Across the five evaluation suites, the four recovery levels exhibit an inherently imbalanced distribution, with higher-difficulty failures occurring less frequently. This distribution emerges naturally from real-world model executions, and we therefore preserve it without manual rebalancing to faithfully reflect the empirical failure distribution. Notably, the pro-

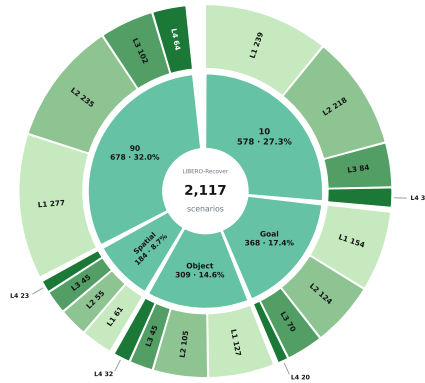


Figure 5: Distribution of LIBERO-Recover scenarios across task suites and recovery difficulty levels, including LIBERO-Spatial, LIBERO-Object, LIBERO-Goal, LIBERO-10 (LIBERO-100) and LIBERO-90 (LIBERO-100).

portion of Post-Task-Failure cases increases with recovery difficulty, as models lacking recovery capability tend to repeatedly retry failed actions, often leading to progressively more severe consequences.

4 DATASETS AND EVALUATION METRICS

4.1 LIBERO-RECOVER DATASET

To improve failure recovery, four human teleoperators collected 3,184 recovery trajectories across 413 failure scenarios using a SpaceMouse, uniformly sampled from the five task suites. These trajectories are used for model fine-tuning. Finetuned models are used to re-generate failure scenarios.

4.2 EVALUATION METRICS

To assess models’ ability to handle failures, we propose the following metrics for evaluation:

Recovery Success Rate (RSR), which measures whether a policy can successfully complete the original task after entering a failure state. For N failure scenarios, we define

$$\text{RSR} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\mathcal{G}(s_T^i, l_i) = 1], \quad (12)$$

where s_T^i denotes the terminal state of the recovered episode. A higher RSR indicates stronger failure recovery capability. To characterize the effect of recovery difficulty, we additionally report RSR_{L_1} , RSR_{L_2} , RSR_{L_3} , and RSR_{L_4} for the four recovery levels.

Recovery Degradation (RD). To quantify the execution capability lost after a failure, we divide each episode into three phases: *Pre-Failure*, *During-Failure*, and *Post-Failure*. Let M_{pre} and M_{post} denote the execution performance before and after failure. We define *Recovery Degradation (RD)* as

$$\text{RD} = 1 - \frac{M_{\text{post}} + \epsilon}{M_{\text{pre}} + \epsilon}, \quad (13)$$

where ϵ ensures numerical stability. Lower RD indicates better retention of execution capability, while higher RD indicates greater degradation.

Recovery Consistency (RC), which measures the stability of recovery across different failure states within the same task. For each task k , let R_k denote its recovery success rate across all associated failure scenarios. We define

$$\text{RC} = 1 - \text{Std}_k(R_k), \quad (14)$$

where $\text{Std}_k(\cdot)$ denotes the standard deviation of task-level recovery success rates. Higher RC indicates more consistent recovery across failure states.

5 EXPERIMENTS

5.1 EXPERIMENTS SETUP

We evaluate six representative models: OpenVLA-OFT Kim et al. (2024), π_0 -Fast Pertsch et al. (2025), GR00T-N1.5 Bjorck et al. (2025), π_0 Black et al. (2024), Wan2-Policy Wan et al. (2025), and Cosmos-Predict2-Policy Alhaija et al. (2025). VLA methods use VLM backbones to encode visual observations and language instructions, while WAM methods generate latent action representations followed by an action head. All models use their official configurations. Following LIBERO Liu et al. (2023), each task is evaluated over 10 trials with slight object-position perturbations, under a time limit of $1.1 \times$ the average human completion time scaled by task difficulty. Initial scene configurations are recorded to ensure consistent starting conditions. Additional experimental details are provided in the appendix.

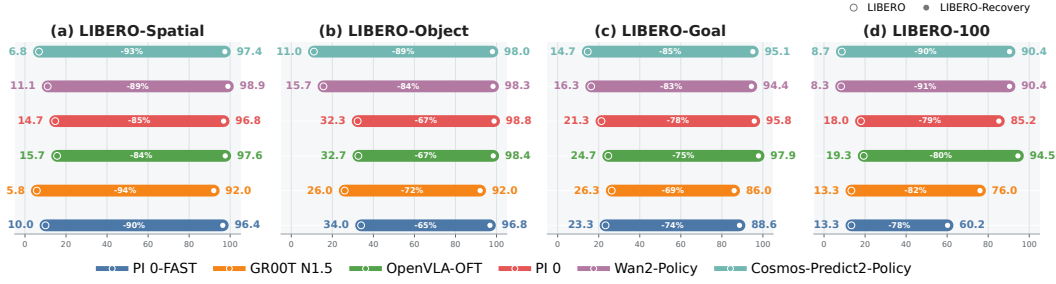


Figure 6: The performance of VLAS and WAMS on LIBERO and LIBERO-Recover.

Table 1: The Multi-level Evaluation on LIBERO-Recover. Darker cells indicate higher recovery success rates. Best results in each difficulty level are shown in bold. PI (PI-0), FAST (PI-0 FAST), GR00T (GR00T N1.5), Wan.(Wan2-Policy) and Cos.(Cosmos-Predict2-Policy).

	LIBERO-Spatial				LIBERO-Object				LIBERO-Goal				LIBERO-100			
Method	L1	L2	L3	L4	L1	L2	L3	L4	L1	L2	L3	L4	L1	L2	L3	L4
PI	40.0	12.5	12.5	6.7	55.7	42.5	24.3	8.8	50.0	22.5	5.7	2.9	38.8	20.0	11.3	0.0
FAST	30.0	12.5	6.7	3.3	50.0	41.3	35.7	11.3	56.3	26.3	5.7	0.0	23.8	11.4	16.3	0.0
GR00T	8.0	12.5	7.5	3.3	47.1	27.5	30.0	2.5	52.5	41.3	5.7	0.0	15.0	20.0	17.5	0.0
OFT	40.0	12.5	12.5	9.3	55.7	42.5	24.3	8.8	56.3	31.3	8.6	4.3	38.8	20.0	11.3	0.0
Wan.	16.0	4.0	12.0	12.5	10.0	32.5	20.0	0.0	33.8	21.3	4.3	2.9	11.3	7.1	13.8	0.0
Cos.	8.0	4.0	14.0	0.0	11.4	23.8	8.6	0.0	31.3	18.8	5.7	0.0	11.3	11.4	11.3	0.0

5.2 MAIN RESULTS

5.2.1 MULTI-LEVEL EVALUATION

Key Finding 1: Success in predefined scenarios does not guarantee robust generalization to real failures. As shown in Fig. 10, performance on predefined scenarios does not reliably reflect embodied models’ capabilities under real execution. All models suffer over 50% performance drops when exposed to naturally occurring failures. More importantly, performance advantages on standard benchmarks do not consistently transfer to failure scenarios. For example, Wan2-Policy outperforms GR00T-N1.5 by +14.40% on LIBERO-100, yet falls -5.0% behind it under corresponding failures. Similarly, π_0 and π_0 -Fast exhibit substantial performance reversals on LIBERO-Spatial. These results demonstrate that standard benchmark rankings are poor predictors of robustness to real execution failures and recovery capability.

Key Finding 2: Existing manipulation models excel at local correction but struggle with state recovery. All models achieve substantially higher success rates on L_1/L_2 failures than on L_3/L_4 , with performance consistently decreasing as recovery difficulty increases. This progressive degradation suggests that current manipulation models can locally adjust their immediate actions but struggle to reason about and recover the underlying task state after failure. L_1 and L_2 can often be addressed by adapting the next action based on visual observations, whereas L_3 and L_4 require structured reasoning about changes in object states, spatial relations, and interaction dependencies, as well as sufficient scene memory. The substantial gap between L_2 and L_3 therefore reveals a transition from *action-level correction* to *state-level recovery*, which remains a major limitation of existing manipulation models.

5.2.2 MULTI-LABEL EVALUATION

Key Finding 3: Failures induce compounding distribution shifts that progressively degrade model performance. All evaluated manipulation models exhibit substantial performance degradation after failure across all four LIBERO suites. As shown in Fig. 7, averaged across models, the success rate drops from 15.0% to 6.7% on LIBERO-Spatial, 35.5% to 4.0% on LIBERO-Goal, 38.3% to 5.0% on LIBERO-Object, and 26.2% to only 0.3% on LIBERO-100, corresponding to recovery degradation rates of 54.0%, 82.0%, 87.0%, and 100.0%, respectively. Notably, performance further decreases from the failure stage to the post-failure stage, suggesting that repeated unsuccessful interactions may progressively drive the execution into increasingly unfamiliar states. These

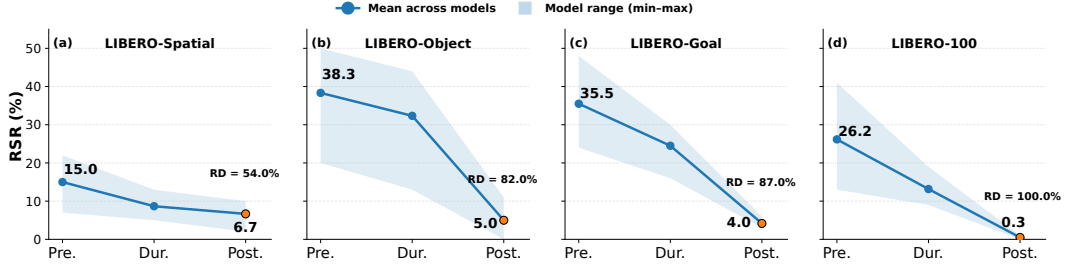


Figure 7: Temporal Manifestation of Failures (RD) in VLAs and WAMs on LIBERO-Recover.

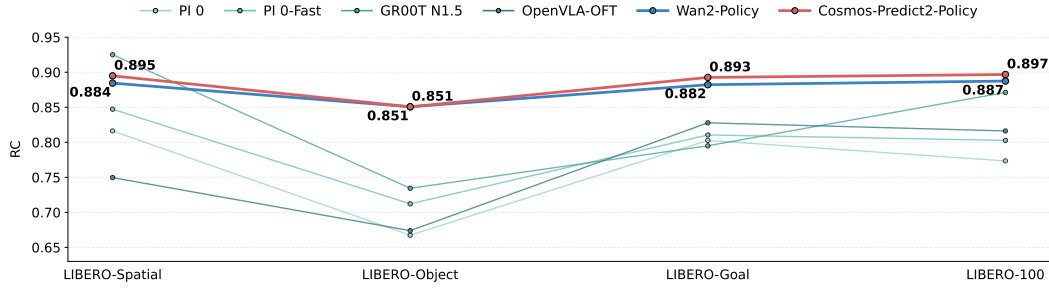


Figure 8: The RC performance of VLAs and WAMs on LIBERO-Recover.

results indicate that failure is not merely a transient disturbance, but can induce a compounding distribution shift that substantially impairs subsequent task execution.

Key Finding 4: WAM policies exhibit more stable recovery. WAM policies consistently achieve higher Recovery Consistency (RC) than conventional VLAs, with Cosmos-Predict2-Policy and Wan2-Policy reaching 0.8837 and 0.8698, respectively, higher compared with conventional VLAs as shown in Fig. 8. A possible explanation is the difference in training objectives: conventional VLAs primarily learn action imitation from successful trajectories, whereas world-model-based policies explicitly model action-conditioned state transitions, encouraging the model to reason about how actions affect the current state and future outcomes. Such a transition-aware representation may provide a stronger inductive bias for adapting to unexpected failure states, leading to more consistent recovery behavior.

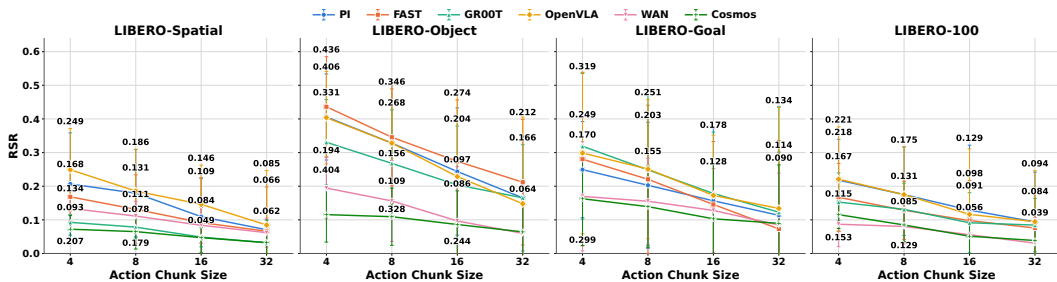


Figure 9: The impact of Chunk Size on Model Recovery Capability.

5.2.3 HOW TO ENHANCE MODEL FAILURE HANDLING?

Key Finding 5: Smaller action chunks improve recovery by enabling more frequent state feedback. Across all four LIBERO subsets, recovery success consistently degrades as the action chunk size increases from 4 to 32, with the most substantial drop generally occurring at larger chunk sizes. This trend is shared across different policies, indicating that recovery performance is highly sensitive to the temporal granularity of action execution. Longer action chunks commit the robot to a

sequence of actions before the next policy update, making it harder to correct deviations caused by failures. In contrast, smaller chunks enable more frequent feedback and action adjustment, allowing the policy to better track the evolving post-failure state. These results suggest that effective failure recovery requires not only a capable policy, but also sufficiently fine-grained closed-loop control.

Key Finding 6: Recovery training does not effectively transfer to failures encountered during standard task execution. Standard LIBERO evaluation is not failure free: even when starting from valid initial states, policies can make mistakes during execution and consequently fail to complete the task. This raises a natural question: can training on failure-recovery scenarios teach a policy to recover from failures that arise during standard task execution, thereby improving its overall LIBERO success rate? To examine this, we jointly train the policies on the original LIBERO training data and LIBERO-Recover data, and evaluate them on both standard LIBERO tasks and LIBERO-Recover scenarios.

Interestingly, the results do not support this expectation. Adding LIBERO-Recover data consistently improves performance on LIBERO-Recover, but produces little or even negative change on standard LIBERO. For GR00T N1.5, mixed training improves the average recovery success rate from 17.8% to 21.2%, while the average LIBERO success rate only increases from 86.5% to 87.2%. For OpenVLA-OFT, recovery success increases from 20.8% to 25.4%, whereas the average LIBERO success rate slightly decreases from 97.1% to 96.6%. Thus, the recovery data clearly provides useful supervision for the recovery scenarios on which it is evaluated, yet this learned recovery capability provides little benefit when failures arise naturally during standard LIBERO execution.

This result reveals a failure-recovery transfer gap. Exposing a policy to failure states is not sufficient to make it reliably recover from its own execution errors during ordinary task execution. ***This suggests that effective recovery may depend not only on learning how to act from a failed state, but also on recognizing and responding to failures that emerge online during the policy’s own execution trajectory.***

Key Finding 7: Temporal context improves failure recovery. Providing the task initial frame as temporal context consistently improves recovery performance across both models, with the largest gains observed on more challenging task settings. For GR00T N1.5, temporal context improves the overall recovery success rate from 17.8% to 18.9%. The effect is substantially stronger for OpenVLA-OFT, whose average recovery success increases from 20.8% to 26.8%, including a notable improvement on LIBERO-100 from 10.3% to 24.7% and on LIBERO-Goal from 24.6% to 30.0%. These improvements suggest that recovery benefits from access to the task’s temporal context rather than relying solely on the current failure state. The initial frame provides a reference for comparing the current scene against the intended task configuration, allowing the policy to identify state deviations and infer what has been disrupted by the failure. This indicates that effective failure recovery requires not only understanding the current state, but also reasoning about how the current state differs from the expected state.

Key Finding 8: Failure recovery occurs in a more concentrated state distribution. The KDE visualization reveals that LIBERO-Recover exhibits a more concentrated state distribution than the original LIBERO data, with several distinct high-density regions, while the original data covers a more dispersed workspace. This suggests that failure recovery is concentrated around a limited number of critical states where failures are more likely to occur, such as grasping, placing, and ob-

Table 2: Performance on LIBERO and LIBERO-Recover after training with mixed LIBERO-Recover expert data.

LIBERO				
Method	LIBERO-Spatial	LIBERO-Object	LIBERO-Goal	LIBERO-100
GR00T N1.5	0.920	0.920	0.860	0.760
GR00T N1.5 (Combined)	0.931	0.905	0.882	0.770
OpenVLA-OFT	0.976	0.984	0.979	0.945
OpenVLA-OFT (Combined)	0.982	0.994	0.970	0.916

LIBERO-Recover				
Method	LIBERO-Spatial	LIBERO-Object	LIBERO-Goal	LIBERO-100
GR00T N1.5	0.057	0.260	0.263	0.133
GR00T N1.5 (Combined)	0.116	0.262	0.264	0.206
OpenVLA-OFT	0.156	0.326	0.246	0.103
OpenVLA-OFT (Combined)	0.178	0.333	0.270	0.236

Table 3: Impact of temporal prompts on model recovery capability.

LIBERO-Recover				
Method	LIBERO-Spatial	LIBERO-Object	LIBERO-Goal	LIBERO-100
GR00T N1.5	0.057	0.260	0.263	0.133
GR00T N1.5 (temporal)	0.066	0.260	0.264	0.166
OpenVLA-OFT	0.156	0.326	0.246	0.103
OpenVLA-OFT (temporal)	0.193	0.333	0.300	0.247

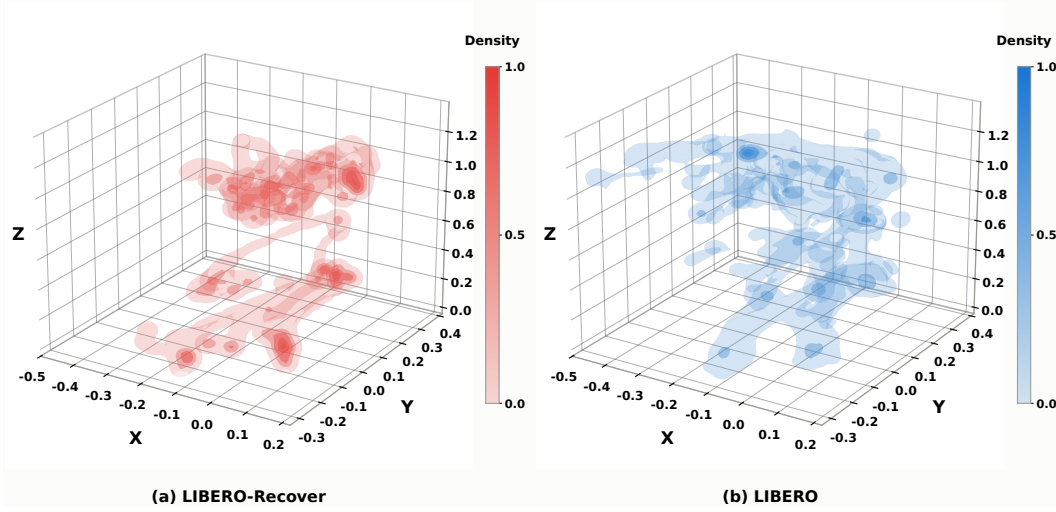


Figure 10: Distribution visualization of training trajectories on LIBERO-Recover and LIBERO.

ject interaction. In contrast, standard demonstrations cover a broader range of successful execution states. This distributional difference indicates that recovery data provides concentrated information around critical state-transition regions that are underrepresented in standard demonstrations, highlighting the need to explicitly model and evaluate failure recovery.

6 CONCLUSION

We introduced **LIBERO-Recover**, a benchmark for evaluating failure recovery in robotic manipulation models. By collecting naturally occurring failures from embodied model executions, LIBERO-Recover evaluates whether models can understand disrupted states, adapt their actions, and recover to complete the original task. Experiments across representative VLA and WAM models reveal a substantial gap between standard task success and failure recovery, with performance degrading sharply as recovery requires increasingly deeper state reasoning. Our analysis further shows that finer-grained action execution and temporal context can improve recovery, while current recovery training provides limited transfer to failures encountered during standard task execution. We hope LIBERO-Recover will encourage future research toward more robust and failure-aware embodied agents.

REFERENCES

- Hassan Abu Alhaija, Jose Alvarez, Maciej Bala, Tiffany Cai, Tianshi Cao, Liz Cha, Joshua Chen, Mike Chen, Francesco Ferroni, Sanja Fidler, et al. Cosmos-transfer1: Conditional world generation with adaptive multimodal control. *arXiv preprint arXiv:2503.14492*, 2025.
- Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Qingwen Bu, Yanting Yang, Jisong Cai, Shenyuan Gao, Guanghui Ren, Maoqing Yao, Ping Luo, and Hongyang Li. Univla: Learning to act anywhere with task-centric latent actions. *arXiv preprint arXiv:2505.06111*, 2025.
- Tianxing Chen, Zanzin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and bench-

-
- mark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.
- Rongxu Cui, Zongzheng Zhang, Jingrui Pang, Haohan Chi, Jinbang Guo, Saining Zhang, Shaoxuan Xie, Xin Jin, Yao Mu, Jiaolong Yang, et al. Libero-safety: A comprehensive benchmark for physical and semantic safety in vision-language-action models. *arXiv preprint arXiv:2606.23686*, 2026.
- Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, et al. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. *arXiv preprint arXiv:2412.14803*, 2024.
- Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- Stephen James, Zicong Ma, David Rovick Arrojo, and Andrew J Davison. Rlbench: The robot learning benchmark & learning environment. *IEEE Robotics and Automation Letters*, 5(2):3019–3026, 2020.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, et al. Cosmos policy: Fine-tuning video models for visuomotor control and planning. *arXiv preprint arXiv:2601.16163*, 2026.
- Xuanlin Li, Kyle Hsu, Jiayuan Gu, Karl Pertsch, Oier Mees, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, et al. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- Jiaming Liu, Mengzhen Liu, Zhenyu Wang, Pengju An, Xiaoqi Li, Kaichen Zhou, Senqiao Yang, Renrui Zhang, Yandong Guo, and Shanghang Zhang. Robomamba: Efficient vision-language-action model for robotic reasoning and manipulation. *Advances in Neural Information Processing Systems*, 37:40085–40110, 2024.
- Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022.
- Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. *arXiv preprint arXiv:2406.02523*, 2024.
- Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwei Yu, Haiming Zhao, Jianxiao Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.
- Guodong Wang, Chenkai Zhang, Qingjie Liu, Jinjin Zhang, Jiancheng Cai, Junjie Liu, and Xinmin Liu. Libero-x: Robustness litmus for vision-language-action models. *arXiv preprint arXiv:2602.06556*, 2026.

Ruijie Zheng, Jing Wang, Scott Reed, Johan Bjorck, Yu Fang, Fengyuan Hu, Joel Jang, Kaushil Kundalia, Zongyu Lin, Loic Magne, et al. Flare: Robot learning with implicit world modeling. *arXiv preprint arXiv:2505.15659*, 2025.

Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.

Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. *arXiv preprint arXiv:2504.02792*, 2025.

A APPENDIX

You may include other additional sections here.